

Verity — reshaped

The pitch: AO made coding agents parallel because code has a merge gate. Verity is the merge gate for everything else — and we prove it by shipping a reconciliation that closes end to end on top of it.

Same name, same core insight, three changes: the agent is real, one workflow ships end to end, and AO is the thesis rather than an afterthought.

The one sentence for the README

Git isolates, CI verifies, review merges — that's why agent orchestration works for code and nowhere else. Verity is the CI for non-code agent work. We built it, then ran a full bank reconciliation on top of it to prove it holds.

That sentence does three jobs at once: it's an AO roadmap argument for Prateek, a controls-and-oversight argument for Maximor, and a plain track-fit statement for the rubric.

Architecture

AO for code	Verity
GitHub issue	one exception (unmatched line, short pay, FX variance, duplicate)
worker + isolated worktree	one agent per exception, isolated context
the PR	proposed journal entry + evidence chain
CI	Verity's control engine
CI failure routed back to the agent	blocked control → structured feedback → agent repairs → resubmits
human merges	controller approves; posting is written
accumulated CI checks	guardrails that write themselves

Three autonomy levels stay exactly as you had them: Auto (evidenced, controls pass, executes), Review (proposes, human approves), Escalate (ambiguous or high-risk, agent stops).

Controls: build three real ones. Evidence completeness (every claim cites a source document), amount tolerance, and FX source validity. Roadmap the other four in the README. Three working controls beat seven stubs.

What gets built for real, and what doesn't

Real, non-negotiable:

- A live LLM agent loop with three tools — fetch invoice, fetch bank line, fetch FX rate — proposing against deliberately messy synthetic data. It must be capable of failing in ways you didn't script.
- The control engine and the block → feedback → repair → resubmit cycle.
- Pattern → guardrail proposal → one human approval → replay of past cases.
- One reconciliation running end to end: ingest, match, exception queue, controller approval, posting written.

Cheap: hash-chained log, twenty lines, one sentence in the demo. It's a 10% category. Do not spend two hours on it.

Not built: ERP connectors, auth, Dodo, multi-tenancy, the other four controls, any second workflow.

Sponsors, maximized where they score

- **AO — 25% of the rubric, and your thesis.** Not just your build tool. Run genuinely parallel workers from minute one; the kanban with real branches and PRs is your evidence. In the README, name Verity as the verifier slot AO's plugin architecture doesn't have. That's a roadmap contribution, not a submission.
- **Neatlogs — your evidence layer, free.** Their detections are rules over spans and their evals are human review campaigns with rating forms and batch scores. Point them at your control-block events and the "evidence of measurable results" requirement is satisfied by a real dashboard. Their founder is a co-host who will see your traces.
- **TensorMux** — meters inference, so cost-per-exception before and after guardrails is a measured number rather than a claim.
- **Maximor** — speak their vocabulary: preparer/reviewer, versioned policy IDs, audit-ready. Their team is in the Discord for 30 hours.

- **Dodo** — zero rubric points. Skip it entirely.
-

The demo, 5 minutes

1. **20s** — the merge-gate table. One slide.
2. **40s** — reconciliation running. Clean lines auto-clear. 14 exceptions become 14 workers.
3. **60s** — a *live* agent proposes an entry with an uncited FX rate. Verity blocks: rate not sourced from an approved provider. Agent receives structured feedback, repairs, resubmits. **This is your CI moment and almost nobody else will have one.**
4. **45s** — escalation, controller approves in one click, evidence chain and hash visible.
5. **60s** — Verity: this FX-mismatch pattern has appeared six times. Proposes a guardrail. One approval. Replay: all six now auto-caught. **Then the move nobody else makes — one case the guardrail should *not* catch passes through cleanly.** You checked your own auto-generated rule for false positives, on camera.
6. **30s** — new identical case arrives, blocked instantly, zero human time. The close: one approval eliminated a recurring failure class permanently.
7. **45s** — AO kanban, real parallel sessions, how the fleet built this.

Open the video on the tagline: **"Don't trust the agent's confidence. Trust what passed."**

Team split, three parallel AO workers

- **A** — synthetic data with seeded exception classes, control engine, hash log
- **B** — the live agent loop, tools, repair cycle, pattern detection and replay
- **C** — exception queue and approval UI, Neatlogs instrumentation, demo capture

Rotate sleep. Do not all crash at once.

Clock (Lagos time, ends 23:00 Sunday)

Hours 0–2: lock spec, generate data, AO sessions live immediately. To hour 8: control engine plus agent loop, first real block. To hour 14: reconciliation closing end to end. To hour 22: pattern → guardrail → replay, including the false-positive case. To hour 26: Neatlogs, metrics, README. To hour 28: demo video. Last two hours: buffer, submit early.

Numbers on screen

Auto-clear rate before and after guardrail learning. Exceptions remaining. Controller minutes spent. Cost per exception. Zero out-of-policy postings.

The scope fence I'm holding you to

If at hour 14 someone says "let's add AP too, to show the platform generalizes" — no. The generalization claim lives in three README paragraphs and one slide. One workflow, three controls, one failure class retired on camera.

Internships

Different game from the prize, run in parallel. Introduce the team in the Discord now with the merge-gate thesis in two sentences — that framing is memorable and nobody else will have it. Ask the Maximor founders one specific question about how they classify exceptions in practice, and use their answer in your exception taxonomy, then say so in the demo. Post your passes tagging AO. Devpost says internships come from **Maximor and AO** both, so the AO argument is working two doors at once.

Go register all three of you on Devpost and start the AO sessions. The clock is already running.